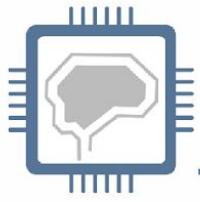


***Inteligência Artificial
e Computacional***

Disciplina: Linguagens de Programação para Inteligência Artificial

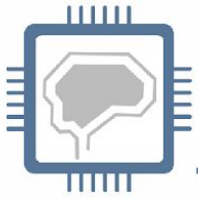
Aula 2: Comandos básicos da linguagem Python - parte II

Profa.: Sabrina de A. Silveira



Estruturas de repetição

As estruturas de repetição, também chamadas de loop, são muito úteis para que um bloco de instruções seja repetido diversas vezes, sem que seja necessário escrever tais instruções repetidamente.



While 1

O loop while, ou melhor, as instruções dentro ou no escopo do loop while, são executadas enquanto uma determinada condição é verdadeira.

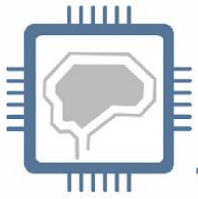
Inicializa a variável

current_number com 1



```
current_number = 1
while current_number <= 5:
    print(current_number)
    # a linha abaixo é igual a current_number = current_number + 1
    current_number += 1
```

```
1
2
3
4
5
```



While 1

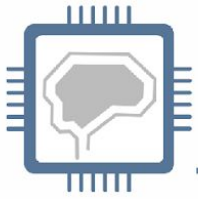
O loop while, ou melhor, as instruções dentro ou no escopo do loop while, são executadas enquanto uma determinada condição é verdadeira.

Inicializa a variável
current_number com 1

As instruções dentro
do while são
executadas 5 vezes,
para contar de 1 a 5.

```
current_number = 1
while current_number <= 5:
    print(current_number)
    # a linha abaixo é igual a current_number = current_number + 1
    current_number += 1
```

1
2
3
4
5



While 1

O loop while, ou melhor, as instruções dentro ou no escopo do loop while, são executadas enquanto uma determinada condição é verdadeira.

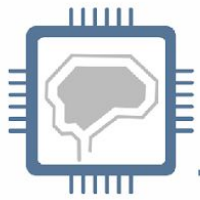
Inicializa a variável
current_number com 1

As instruções dentro
do while são
executadas 5 vezes,
para contar de 1 a 5.

Saída

```
current_number = 1
while current_number <= 5:
    print(current_number)
    # a linha abaixo é igual a current_number = current_number + 1
    current_number += 1
```

```
{
  1
  2
  3
  4
  5
}
```



While 2

Um trecho de código pode também ser executado repetidas vezes até que o usuário digite um valor pré-definido, que cause o fim da execução do programa.

```
prompt = "\nDiga-me alguma coisa e eu vou repetir:"  
prompt += "\nEntre 'pare' para terminar o programa. "  
message = ""  
while message != 'pare':  
    message = input(prompt)  
    print(message)
```

```
Diga-me alguma coisa e eu vou repetir:  
Entre 'pare' para terminar o programa.
```

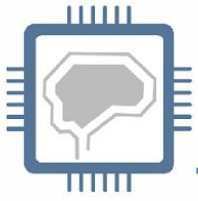
```
teste
```

```
teste
```

```
Diga-me alguma coisa e eu vou repetir:  
Entre 'pare' para terminar o programa.
```

```
pare
```

```
pare
```

While 2

Esse bloco é repetido até que o usuário digite **pare**

```
prompt = "\nDiga-me alguma coisa e eu vou repetir:"  
prompt += "\nEntre 'pare' para terminar o programa. "  
message = ""  
while message != 'pare':  
    { message = input(prompt)  
      print(message)
```

```
Diga-me alguma coisa e eu vou repetir:  
Entre 'pare' para terminar o programa.
```

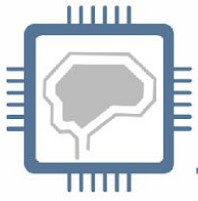
teste

teste

```
Diga-me alguma coisa e eu vou repetir:  
Entre 'pare' para terminar o programa.
```

pare

pare



While 2

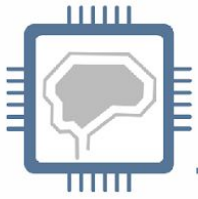
Esse bloco é repetido até que o usuário digite **pare**

Observe o exemplo de saída para as entradas **teste** e **pare**

```
prompt = "\nDiga-me alguma coisa e eu vou repetir:"  
prompt += "\nEntre 'pare' para terminar o programa. "  
message = ""  
while message != 'pare':  
    { message = input(prompt)  
      print(message)
```

```
Diga-me alguma coisa e eu vou repetir:  
Entre 'pare' para terminar o programa.  
teste  
teste
```

```
Diga-me alguma coisa e eu vou repetir:  
Entre 'pare' para terminar o programa.  
pare  
pare
```

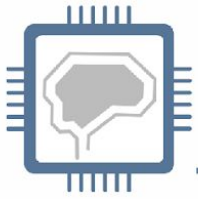
While 3

Em situações onde há vários eventos diferentes que podem terminar a execução do programa, podemos criar uma variável **flag** que será avaliada para decidir se o programa deve continuar ou não.

```
prompt = "\nDiga-me alguma coisa e eu vou repetir:"
prompt += "\nEntre 'pare' para terminar o programa. "
active = True
while active:
    message = input(prompt)
    if message == 'pare':
        active = False
    else:
        print(message)
```

```
Diga-me alguma coisa e eu vou repetir:
Entre 'pare' para terminar o programa. Olá
Olá
```

```
Diga-me alguma coisa e eu vou repetir:
Entre 'pare' para terminar o programa. pare
```



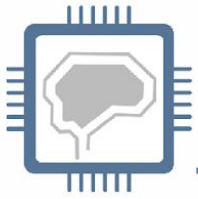
While 3

As instruções dentro desse bloco são repetidas enquanto a variável **active** for True.

```
prompt = "\nDiga-me alguma coisa e eu vou repetir:"  
prompt += "\nEntre 'pare' para terminar o programa. "  
active = True  
while active:  
    message = input(prompt)  
    if message == 'pare':  
        active = False  
    else:  
        print(message)
```

```
Diga-me alguma coisa e eu vou repetir:  
Entre 'pare' para terminar o programa. Olá  
Olá
```

```
Diga-me alguma coisa e eu vou repetir:  
Entre 'pare' para terminar o programa. pare
```



While 3

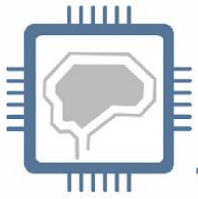
As instruções dentro desse bloco são repetidas enquanto a variável **active** for True.

Quando o usuário digita **pare**, a variável **active** recebe False.

```
prompt = "\nDiga-me alguma coisa e eu vou repetir:"
prompt += "\nEntre 'pare' para terminar o programa. "
active = True
while active:
    message = input(prompt)
    if message == 'pare':
        active = False
    else:
        print(message)
```

```
Diga-me alguma coisa e eu vou repetir:
Entre 'pare' para terminar o programa. Olá
Olá
```

```
Diga-me alguma coisa e eu vou repetir:
Entre 'pare' para terminar o programa. pare
```



While 3

As instruções dentro desse bloco são repetidas enquanto a variável **active** for True.

Quando o usuário digita **pare**, a variável **active** recebe False.

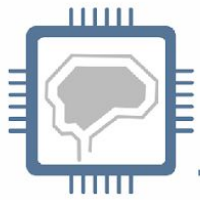
Na próxima vez que o loop while testar a variável **active**, ela será False e não entrará no bloco while.

Observe o exemplo de saída para as entradas **Olá** e **pare**

```
prompt = "\nDiga-me alguma coisa e eu vou repetir:"  
prompt += "\nEntre 'pare' para terminar o programa. "  
active = True  
while active:  
    message = input(prompt)  
    if message == 'pare':  
        active = False  
    else:  
        print(message)
```

```
Diga-me alguma coisa e eu vou repetir:  
Entre 'pare' para terminar o programa. Olá  
Olá
```

```
Diga-me alguma coisa e eu vou repetir:  
Entre 'pare' para terminar o programa. pare
```

While 4

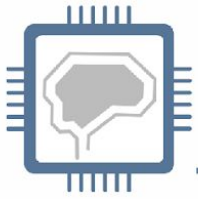
A estrutura de repetição while pode ser utilizada com o comando break.

Este comando possibilita que o loop seja interrompido sem rodar nenhuma das instruções restantes dentro do loop

```
prompt = "\nInforme o nome de uma cidade que você visitou:"
prompt += "\n(Digite 'pare' quando tiver informado a cidade.) "
while True:
    city = input(prompt)
    if city == 'pare':
        break
    else:
        print(f"Eu gostaria de visitar {city.title()}!")
```

```
Informe o nome de uma cidade que você visitou:
(Digite 'pare' quando tiver informado a cidade.) Praga
Eu gostaria de visitar Praga!
```

```
Informe o nome de uma cidade que você visitou:
(Digite 'pare' quando tiver informado a cidade.) pare
```



While 4

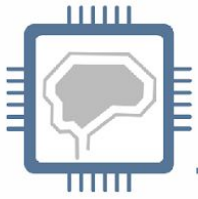
Quando a variável **city** recebe o valor **pare**, executa o break e sai do while

Observe o exemplo de saída para as entradas **Praga** e **pare**

```
prompt = "\nInforme o nome de uma cidade que você visitou:"
prompt += "\n(Digite 'pare' quando tiver informado a cidade.) "
while True:
    city = input(prompt)
    if city == 'pare':
        break
    else:
        print(f"Eu gostaria de visitar {city.title()}!")
```

```
Informe o nome de uma cidade que você visitou:
(Digite 'pare' quando tiver informado a cidade.) Praga
Eu gostaria de visitar Praga!
```

```
Informe o nome de uma cidade que você visitou:
(Digite 'pare' quando tiver informado a cidade.) pare
```

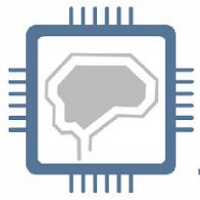
While 5

Invés de sair do while sem executar as demais instruções com o comando **break**, você pode usar o **continue**.

O comando **continue** retorna ao início do loop com base no resultado de um teste condicional.

```
current_number = 0
while current_number < 10:
    current_number += 1
    if current_number % 2 == 0:
        continue
    print(current_number)
```

1
3
5
7
9

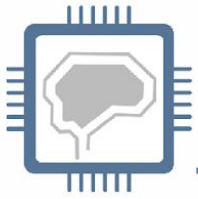


While 5

As instruções dentro desse bloco são repetidas enquanto a variável `current_number < 10`

```
current_number = 0
while current_number < 10:
    current_number += 1
    if current_number % 2 == 0:
        continue
    print(current_number)
```

1
3
5
7
9



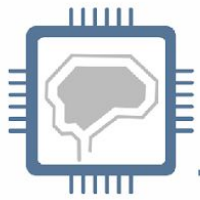
While 5

As instruções dentro desse bloco são repetidas enquanto a variável `current_number < 10`

Se o resto da divisão por 2 é 0, ou seja se o número é par, não faz mais nada e retorna ao início do loop.

```
current_number = 0
while current_number < 10:
    current_number += 1
    if current_number % 2 == 0:
        continue
    print(current_number)
```

1
3
5
7
9



While 5

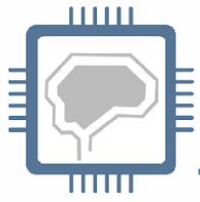
As instruções dentro desse bloco são repetidas enquanto a variável `current_number < 10`

Se o resto da divisão por 2 é 0, ou seja se o número é par, não faz mais nada e retorna ao início do loop.

Assim, são impressos apenas os números ímpares.

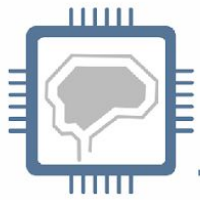
```
current_number = 0
while current_number < 10:
    current_number += 1
    if current_number % 2 == 0:
        continue
    print(current_number)
```

1
3
5
7
9



Estruturas de repetição

É importante ter cuidado com os loops infinitos. Toda estrutura de repetição deve parar de executar em algum momento. Caso isso não aconteça, você deve estar em uma situação em que a condição de parada nunca é satisfeita.



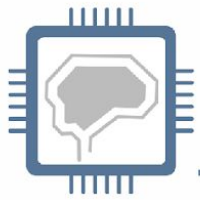
For 1

A estrutura de repetição for é frequentemente utilizada para iterar (ou percorrer) uma sequência de elementos (pode ser um vetor, uma lista, um string, dentre outros).

Aqui o **for** é usado para percorrer uma lista de frutas

```
fruits = ["banana", "uva", "abacaxi", "melancia"]  
for x in fruits:  
    print(x)
```

```
banana  
uva  
abacaxi  
melancia
```



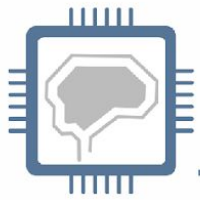
For 1

A estrutura de repetição for é frequentemente utilizada para iterar (ou percorrer) uma sequência de elementos (pode ser um vetor, uma lista, um string, dentre outros).

Para cada fruta da lista →

```
fruits = ["banana", "uva", "abacaxi", "melancia"]  
for x in fruits:  
    print(x)
```

```
banana  
uva  
abacaxi  
melancia
```



For 1

A estrutura de repetição for é frequentemente utilizada para iterar (ou percorrer) uma sequência de elementos (pode ser um vetor, uma lista, um string, dentre outros).

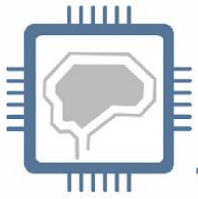
Para cada fruta da lista

Imprime a fruta



```
fruits = ["banana", "uva", "abacaxi", "melancia"]  
for x in fruits:  
    print(x)
```

```
banana  
uva  
abacaxi  
melancia
```



For 1

A estrutura de repetição for é frequentemente utilizada para iterar (ou percorrer) uma sequência de elementos (pode ser um vetor, uma lista, um string, dentre outros).

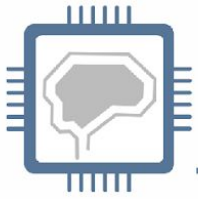
Para cada fruta da lista

Imprime a fruta

```
fruits = ["banana", "uva", "abacaxi", "melancia"]  
for x in fruits:  
    print(x)
```

Observe a saída

```
{  
  banana  
  uva  
  abacaxi  
  melancia  
}
```



For 2

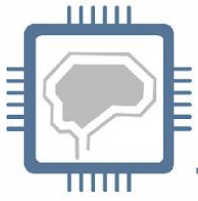
O comando for também pode ser utilizado para percorrer ou iterar sobre os caracteres de um string.

Para cada letra do string



```
for x in "melancia":  
    print(x)
```

m
e
l
a
n
c
i
a



For 2

O comando for também pode ser utilizado para percorrer ou iterar sobre os caracteres de um string.

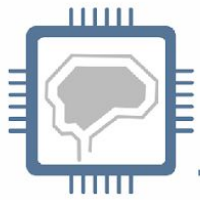
Para cada letra do string

Imprime a letra



```
for x in "melancia":  
    print(x)
```

```
m  
e  
l  
a  
n  
c  
i  
a
```

For 2

O comando for também pode ser utilizado para percorrer ou iterar sobre os caracteres de um string.

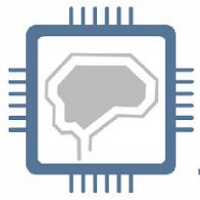
Para cada letra do string

Imprime a letra

```
for x in "melancia":  
    print(x)
```

Observe a saída

{
m
e
l
a
n
c
i
a



For 3

Para iterar um determinado número de vezes com o comando for, podemos utilizar a função range().

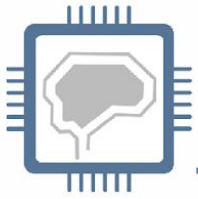
Para x variando de 0 a 4



```
for x in range(5):  
    print(x)
```

O range vai até 5 - 1

0
1
2
3
4



For 3

Para iterar um determinado número de vezes com o comando for, podemos utilizar a função range().

Para x variando de 0 a 4

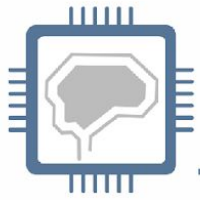
Imprime x



```
for x in range(5):  
    print(x)
```

O range vai até 5 - 1

0
1
2
3
4



For 3

Para iterar um determinado número de vezes com o comando for, podemos utilizar a função range().

O range vai até 5 - 1

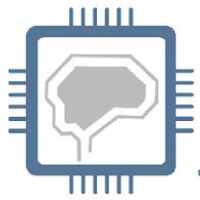
Para x variando de 0 a 4

Imprime x

```
for x in range(5):  
    print(x)
```

Observe a saída

{
0
1
2
3
4



For 4

A função range também pode ser utilizada para especificar os valores de início e fim da repetição.

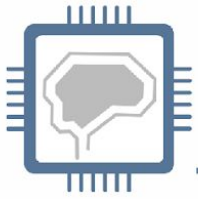
Para x variando de 2 a 5



```
for x in range(2, 6):  
    print(x)
```

O range vai até 6 - 1

2
3
4
5



For 4

A função range também pode ser utilizada para especificar os valores de início e fim da repetição.

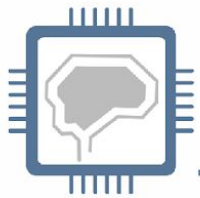
Para x variando de 2 a 5
Imprime x



```
for x in range(2, 6):  
    print(x)
```

O range vai até 6 - 1

2
3
4
5



For 4

A função range também pode ser utilizada para especificar os valores de início e fim da repetição.

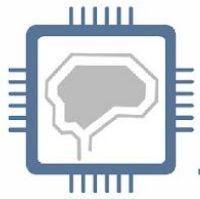
Para x variando de 2 a 5
Imprime x

```
for x in range(2, 6):  
    print(x)
```

O range vai até 6 - 1

Observe a saída

{
2
3
4
5



For 5

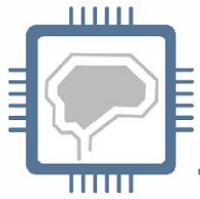
É possível ainda especificar o valor de início e fim da repetição, bem como o passo, que é a variação do valor a cada iteração.

O range vai até 20

Para x variando de 0 a 20, com passo 4 →

```
for x in range(0, 21, 4):  
    print(x)
```

0
4
8
12
16
20



For 5

É possível ainda especificar o valor de início e fim da repetição, bem como o passo, que é a variação do valor a cada iteração.

Para x variando de 0 a 20, com passo 4

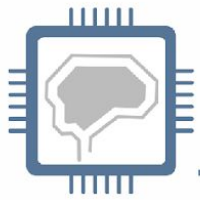
Imprime x



```
for x in range(0, 21, 4):  
    print(x)
```

O range vai até 20

0
4
8
12
16
20



For 5

É possível ainda especificar o valor de início e fim da repetição, bem como o passo, que é a variação do valor a cada iteração.

Para x variando de 0 a 20, com passo 4

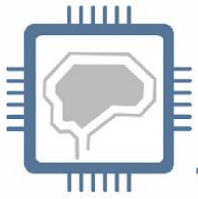
Imprime x

```
for x in range(0, 21, 4):  
    print(x)
```

O range vai até 20

Observe a saída

{
0
4
8
12
16
20



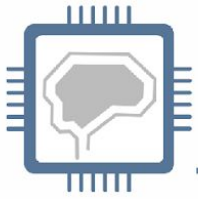
Entrada e saída a partir de arquivos

Através da linguagem de programação Python, podemos fazer a leitura e escrita de arquivos. O primeiro passo para ler ou escrever num arquivo, é abri-lo com a função **open**.

Todo arquivo aberto deve ser fechado. Aqui usamos a função **close**.

```
file1 = open("Teste.txt", "a")  
file1.close()
```

```
file2 = open(r"\Teste.txt", "w+")  
file2.close()
```

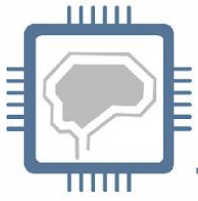


Entrada e saída a partir de arquivos

A função open aceita 6 modos de leitura:

- Somente leitura ('r')
- Leitura e escrita ('r+')
- Somente escrita ('w')
- Leitura e escrita ('w+')
- Somente acrescentar ('a')
- Acrescentar e ler ('a+')

No caso das duas opções para leitura e escrita, a diferença é que ('r+'), dá erro de I/O se o arquivo não existir, enquanto em ('w+') o arquivo é sobrescrito se já existir.



Entrada e saída a partir de arquivos (read 1)

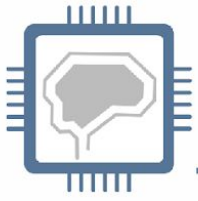
Aqui o arquivo texto é lido de uma só vez com a função **read** e impresso na tela.

A função **close** não é utilizada para fechar o arquivo porque estamos utilizando o **with**, que já se encarrega de fazer o fechamento.

```
with open('caes.txt', 'r') as reader:  
    print(reader.read())
```

Saída

```
Pug  
Jack Russell Terrier  
English Springer Spaniel  
German Shepherd  
Staffordshire Bull Terrier  
Cavalier King Charles Spaniel  
Golden Retriever  
West Highland White Terrier  
Boxer  
Border Terrier
```



Entrada e saída a partir de arquivos (read 2)

Neste programa são lidos 5 bytes (5 caracteres) a cada chamada da função **readline**. Vai lendo de 5 em 5 bytes até atingir o fim da linha.

```
with open('caes.txt', 'r') as reader:  
    print(reader.readline(5))  
    print(reader.readline(5))  
    print(reader.readline(5))  
    print(reader.readline(5))  
    print(reader.readline(5))
```

Saída

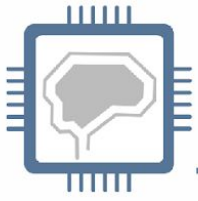
Pug

Jack

Russe

ll Te

rrier



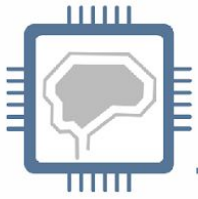
Entrada e saída a partir de arquivos (read 3)

O arquivo pode ainda ser lido inteiro de uma só vez como uma lista utilizando a função **readlines**. Cada linha do arquivo será um elemento da lista.

```
f = open('caes.txt')  
mylist = f.readlines()  
print(mylist)
```

Fragmento da saída

```
['Pug\n', 'Jack Russell Terrier\n', 'English Springer Spaniel\n', 'German Shepherd\n',
```

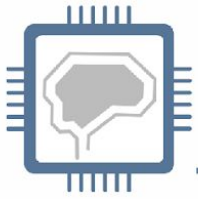
Entrada e saída a partir de arquivos (read 4)

Aqui o arquivo texto é percorrido e impresso linha por linha com o auxílio da função **readline**.

Saída

```
with open('caes.txt', 'r') as reader:
    line = reader.readline()
    # Enquanto há linhas para ler
    while line != '':
        print(line, end='')
        line = reader.readline()
```

Pug
Jack Russell Terrier
English Springer Spaniel
German Shepherd
Staffordshire Bull Terrier
Cavalier King Charles Spaniel
Golden Retriever
West Highland White Terrier
Boxer
Border Terrier



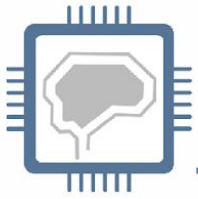
Entrada e saída a partir de arquivos (read 5)

Neste programa um arquivo texto também é lido linha por linha, mas utilizando a função `readlines()`. Ela retorna uma lista onde cada elemento é uma linha do arquivo.

```
with open('caes.txt', 'r') as reader:  
    for line in reader.readlines():  
        print(line, end='')
```

Saída

```
Pug  
Jack Russell Terrier  
English Springer Spaniel  
German Shepherd  
Staffordshire Bull Terrier  
Cavalier King Charles Spaniel  
Golden Retriever  
West Highland White Terrier  
Boxer  
Border Terrier
```



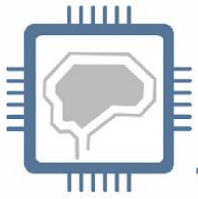
Entrada e saída a partir de arquivos (write 1)

Serão apresentadas 2 alternativas para escrever em arquivo.

O arquivo é lido para uma lista (dogs) onde cada elemento é uma linha do arquivo.

```
with open('caes.txt', 'r') as reader:  
    # dogs é uma lista onde cada elemento é uma linha do arquivo  
    dogs = reader.readlines()
```

```
with open('caes_invertido.txt', 'w') as writer:  
    # escreve os elementos de uma lista com writelines  
    writer.writelines(reversed(dogs))
```



Entrada e saída a partir de arquivos (write 1)

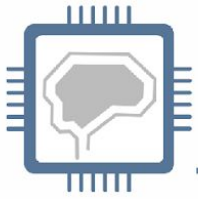
Serão apresentadas 2 alternativas para escrever em arquivo.

O arquivo é lido para uma lista (dogs) onde cada elemento é uma linha do arquivo.

```
with open('caes.txt', 'r') as reader:  
    # dogs é uma lista onde cada elemento é uma linha do arquivo  
    dogs = reader.readlines()
```

```
with open('caes_invertido.txt', 'w') as writer:  
    # escreve os elementos de uma lista com writelines  
    writer.writelines(reversed(dogs))
```

Então um arquivo é escrito utilizando a função `writelines( )`, que escreve os elementos de uma lista no arquivo de saída.



Entrada e saída a partir de arquivos (write 1)

Serão apresentadas 2 alternativas para escrever em arquivo.

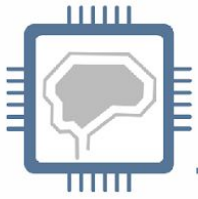
O arquivo é lido para uma lista (dogs) onde cada elemento é uma linha do arquivo.

```
with open('caes.txt', 'r') as reader:  
    # dogs é uma lista onde cada elemento é uma linha do arquivo  
    dogs = reader.readlines()
```

```
with open('caes_invertido.txt', 'w') as writer:  
    # escreve os elementos de uma lista com writelines  
    writer.writelines(reversed(dogs))
```

Então um arquivo é escrito utilizando a função `writelines()`, que escreve os elementos de uma lista no arquivo de saída.

A função `reversed` inverte a ordem dos elementos da lista.



Entrada e saída a partir de arquivos (write 1)

Apresentamos o arquivo de saída referente ao programa:

```
with open('caes.txt', 'r') as reader:
```

```
# dogs é uma lista onde cada elemento é uma linha do arquivo
```

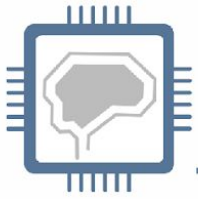
```
dogs = reader.readlines()
```

```
with open('caes_invertido.txt', 'w') as writer:
```

```
# escreve os elementos de uma lista com writelines
```

```
writer.writelines(reversed(dogs))
```

```
Border TerrierBoxer
West Highland White Terrier
Golden Retriever
Cavalier King Charles Spaniel
Staffordshire Bull Terrier
German Shepherd
English Springer Spaniel
Jack Russell Terrier
Pug
```



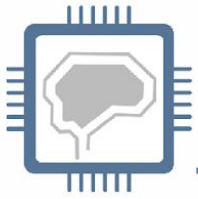
Entrada e saída a partir de arquivos (write 2)

Agora escreveremos mais um arquivo, percorrendo a lista dogs e escrevendo cada um de seus elementos como uma linha do arquivo de saída utilizando a função write ().

O arquivo é lido para uma lista (dogs) onde cada elemento é uma linha do arquivo.

```
with open('caes.txt', 'r') as reader:  
    # dogs é uma lista onde cada elemento é uma linha do arquivo  
    dogs = reader.readlines()
```

```
with open('caes_invertido2.txt', 'w') as writer:  
    # percorre elementos da lista dogs  
    for breed in reversed(dogs):  
        # escreve cada elemento (um string)  
        writer.write(breed)
```



Entrada e saída a partir de arquivos (write 2)

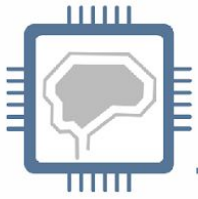
Agora escreveremos mais um arquivo, percorrendo a lista `dogs` e escrevendo cada um de seus elementos como uma linha do arquivo de saída utilizando a função `write ( )`.

O arquivo é lido para uma lista (`dogs`) onde cada elemento é uma linha do arquivo.

```
with open('caes.txt', 'r') as reader:  
    # dogs é uma lista onde cada elemento é uma linha do arquivo  
    dogs = reader.readlines()
```

```
with open('caes_invertido2.txt', 'w') as writer:  
    # percorre elementos da lista dogs  
    for breed in reversed(dogs):  
        # escreve cada elemento (um string)  
        writer.write(breed)
```

Percorre a lista `dogs` escrevendo cada um de seus elementos como uma linha do arquivo.



Entrada e saída a partir de arquivos (write 2)

Apresentamos o arquivo de saída referente ao programa:

```
with open('caes.txt', 'r') as reader:
```

```
# dogs é uma lista onde cada elemento é uma linha do arquivo
```

```
dogs = reader.readlines()
```

```
with open('caes_invertido2.txt', 'w') as writer:
```

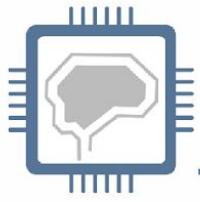
```
# percorre elementos da lista dogs
```

```
for breed in reversed(dogs):
```

```
# escreve cada elemento (um string)
```

```
writer.write(breed)
```

```
Border TerrierBoxer
West Highland White Terrier
Golden Retriever
Cavalier King Charles Spaniel
Staffordshire Bull Terrier
German Shepherd
English Springer Spaniel
Jack Russell Terrier
Pug
```

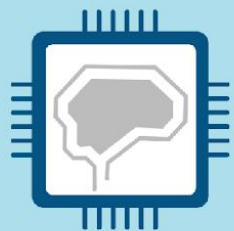


Comandos básicos da linguagem Python

Recomendamos fortemente que o aluno implemente e execute os programas vistos nessa aula, para compreender e exercitar os conceitos na prática.

Os exercícios referentes ao capítulo 2 devem ser implementados para compreensão e fixação dos conteúdos.

Lembre-se: programação precisa de prática!



Inteligência Artificial e Computacional

Profa.: Sabrina de A. Silveira